Proyecto programación orientada a objetos

Estudiantes:

Samuel Marroquín

María Camila Pradilla

John Fredy Gallego Zambrano

Ingeniería de Sistemas / Computación Científica

Proyecto:

Core-Dump: System Recovery

Docente:

Jesús Andrés Hincapié Londoño

Universidad de Medellin

2026-1

1. Descripción del problema

Introducción

El proyecto consiste en el desarrollo de un videojuego de aventura sistémica. El entorno de juego simula el interior de un sistema computacional donde las entidades lógicas han perdido su integridad y el usuario debe restaurarlas. La arquitectura y lógica interna estarán fundamentadas estrictamente en la Programación Orientada a Objetos (POO) utilizando Python. Para cumplir con los estándares del curso en la entrega final, el juego integrará una Interfaz Gráfica de Usuario (GUI) que facilitará la inmersión y la interacción del jugador con el entorno.

Reglas de Negocio y Mecánicas del Juego

El núcleo del juego se rige por las siguientes mecánicas y reglas de interacción:

- Entidades y Actores: El ecosistema del juego contiene entidades con comportamiento activo, como el Jugador, personajes de apoyo (NPCs) y amenazas (Virus). Cada entidad posee estados propios (integridad, energía) que determinan su funcionamiento.
- Exploración y Navegación: El mundo virtual está segmentado en sectores o nodos del sistema, equivalentes a habitaciones o niveles. El progreso requiere que el jugador ejecute un sistema de movimiento entre estos sectores para descubrir áreas dañadas.
- Gestión de Recursos e Inventario: El entorno contiene objetos interactuables vitales para la misión, tales como Scripts, Parches y Llaves. La regla principal de progreso es la manipulación de estos objetos para restaurar el sistema. El jugador deberá administrarlos mediante un sistema de inventario.
- Resolución de Conflictos: Los encuentros con amenazas se manejan mediante un sistema de combate o resolución de conflictos. El éxito en estos encuentros se calcula matemáticamente basándose en los estados actuales del jugador y los recursos que decida utilizar en ese momento.

Objetivos Técnicos

A nivel de desarrollo, el propósito principal es aplicar de forma práctica los cuatro pilares de la POO:

- Abstracción: Modelado de las entidades del mundo virtual en estructuras de datos lógicas.
- Encapsulamiento: Protección de los estados internos de los elementos del juego para evitar modificaciones no autorizadas.
- Herencia: Creación de jerarquías para optimizar la construcción de los distintos elementos del mundo.
- Polimorfismo: Implementación de mecánicas que permitan a distintos elementos del juego reaccionar de forma única ante una misma acción del jugador.

Alcance del Proyecto (MVP y Entrega Final)

- Sistema de movimiento entre nodos de la red.
- Sistema de resolución de conflictos.
- Inventario de objetos y manipulación de recursos.

Interfaz Gráfica de Usuario (GUI): Aunque la lógica interactiva inicial partirá de la recepción y procesamiento de comandos del usuario, la entrega final implementará una interfaz gráfica. Esta reemplazará la consola de texto para brindar retroalimentación visual, clara y en tiempo real sobre el estado del jugador y el entorno.

Justificación Académica

Este proyecto es altamente relevante para la formación en Computación Científica, ya que exige un diseño riguroso de la lógica de negocio. El desarrollo en Python permitirá demostrar un manejo avanzado de colecciones de datos y la lectura y procesamiento de información para la toma de decisiones dinámicas dentro del sistema.

Análisis del problema

Requisitos Funcionales (RF)

A continuación, se definen los requisitos centrados en la jugabilidad y los objetivos del sistema.

ID	Requisito	Descripción	Prototipo de Interfaz (Concepto)
RF01	Navegación de Red	El jugador debe poder trasladarse entre diferentes nodos (sectores) del sistema para localizar fallos.	Un panel central con un grafo de nodos interconectados. Los nodos visitados se iluminan en verde.
RF02	Gestión de Integridad	El sistema debe permitir al usuario recolectar e instalar "Parches" para recuperar puntos de vida del sistema.	Una barra de progreso dinámica en la parte superior que cambia de color según el porcentaje de salud.
RF03	Análisis de Amenazas	El usuario puede escanear una entidad (Virus) para conocer sus debilidades antes de interactuar.	Una ventana emergente (Pop-up) con los atributos del enemigo y una descripción de su comportamiento.
RF04	Cifrado de Puertos (Samuel)	Requisito Innovador: Para entrar a nodos críticos, el usuario debe resolver un sistema de ecuaciones lineales que representa la llave de acceso.	Una cuadrícula de 3×3 donde se ingresan valores numéricos para equilibrar una matriz.
RF05	Evolución Viral (Maria)	Requisito Innovador: Los virus aumentan su complejidad y daño de forma proporcional al tiempo que el jugador tarde en limpiar un sector.	Un contador de "Nivel de Infección" ambiental que distorsiona los colores de la interfaz.

ID	Requisito	Descripción	Prototipo de Interfaz (Concepto)
RF06	Rastreo de Logs (John)	Requisito Innovador: El jugador puede extraer fragmentos de código de sectores dañados para reconstruir habilidades especiales permanentes.	Un visor de "código fuente" donde se seleccionan líneas de texto para ganar bonificadores.

2. Modelo del Mundo (Diagrama de Clases UML)

El diseño se basa en una jerarquía lógica para optimizar la reutilización de código.

- GameObject (Abstracta): Define la identidad base de cualquier elemento en el sistema.
- Actor: Entidades con comportamiento activo e integridad (Jugador, Virus).
- Item: Objetos pasivos con métodos de uso (Parches, Scripts).
- Environment: Representa los nodos de la red y su estado de integridad.
- Engine (Controlador): Gestiona el bucle principal, la instanciación y el flujo de datos.